

Lecture Notes for Neural Networks I



Logistic Regression II

Class Logistics and Agenda

- Last Time: Logistic Regression
- Logistics
 - Grading
- Agenda
 - Logistic Regression
 - Regularization Review
 - Hessian Optimization
 - Demo with Titanic Data
 - Lab Town Hall

Class Overview, by topic

Table Data
Visualization

Numpy, Pandas, Seaborn
Overviews with some in-depth discussion

Linear and
Logistic
Regression

Numpy, Recreate API for Scikit-learn
Detailed mathematics for simple optimization
intuition for advanced optimization

Neural Networks
and Back Prop.

Numpy, Recreate API for DL Frameworks
Detailed mathematics for NN operations

Wide and Deep
Networks

Convolutional
Networks

Sequential
Networks

Large Language
Models (if time)

Hugging Face
Intuition only

Review, Last Time:

Objective
Function

$$l(\mathbf{w}) = \sum_i y^{(i)} \ln (g(\mathbf{x}^{(i)} \cdot \mathbf{w})) + (1 - y^{(i)}) \ln (1 - g(\mathbf{x}^{(i)} \cdot \mathbf{w}))$$

Update
Function

$$\mathbf{w} \leftarrow \mathbf{w} + \frac{\eta}{M} \sum_{i=1}^M (y^{(i)} - g(\mathbf{x}^{(i)} \cdot \mathbf{w})) \cdot \mathbf{x}^{(i)}$$

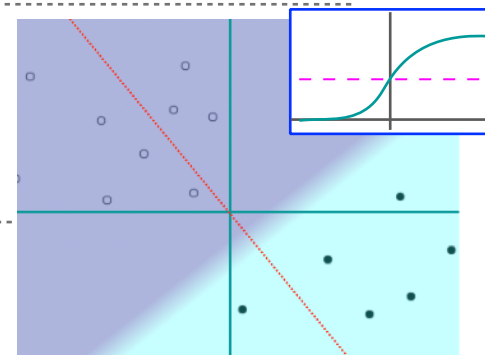
Vectorized
Update

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot \text{mean}(\mathbf{y}_{diff} \odot \mathbf{X})_{columns}$$

Regularization

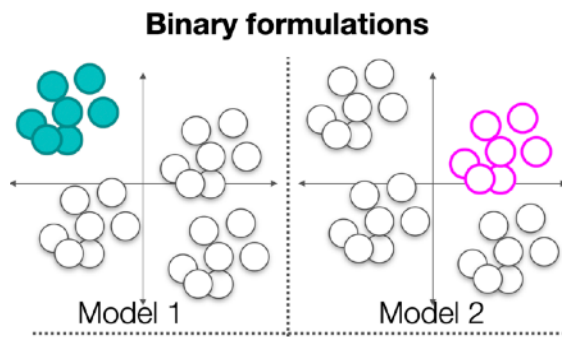
$$l(\mathbf{w}) = l(\mathbf{w})_{prev} - C \cdot \sum_j w_j^2$$

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \left[\nabla l(\mathbf{w})_{prev} - C \cdot 2\mathbf{w} \right]$$



Multi-class
Formulation

One model
per Class



					classes[0 1 2 3]			
					model probabilities			
test data (rows)	0.91	0.0	0.78	0.53	arg max	0	hash	
	0.53	0.5	0.32	0.91	arg max	3	hash	
	0.01	0.6	0.98	0.01	arg max	2	hash	
	0.05	0.83	0.21	0.81	arg max	1	hash	
	0.85	0.02	0.33	0.02	arg max	0	hash	
	0.63	0.11	0.09	0.33	arg max	0	hash	
	0.01	0.32	0.12	0.88	arg max	3	hash	
					M1 M2 M3 M4			

Last Time:

```
%%time
ds = load_iris()
X = ds.data
y = ds.target # note problem is NOT binary anymore, there are three classes!

X_train, X_test, y_train, y_test = train_test_split(X,y, train_size = 0.8, test_size=0.2)

lr = LogisticRegression(0.1,500)
lr.fit(X_train,y_train)
print(lr)

yhat = lr.predict(X_test)
print('Accuracy of: ',accuracy_score(y_test,yhat))
```

```
MultiClass Logistic Regression Object with coefficients:
[[ 0.29773533  0.4533672  1.55246277 -2.41143016 -1.13092625]
 [ 0.38589636  0.31560559 -1.28306232  0.5421915  -0.97321777]
 [-0.81705268 -1.49226693 -1.41357874  2.20417964  2.04839556]]
Accuracy of: 0.9666666666666667
CPU times: user 35.5 ms, sys: 3.66 ms, total: 39.2 ms
Wall time: 37.6 ms
```

Ours

sklearn has some aspects that we do not yet implement:

- regularization (be less overconfident)
- better direction finding for the next step in updating $\mathbf{w}$

```
%%time
from sklearn.linear_model import LogisticRegression as SKLogisticRegression

lr_sk = SKLogisticRegression(solver='lbfgs') # all params default

lr_sk.fit(X_train,y_train)
print(np.hstack((lr_sk.intercept_[:],np.newaxis),lr_sk.coef_)))
yhat = lr_sk.predict(X_test)
print('Accuracy of: ',accuracy_score(y_test,yhat))
```

```
[[ 9.56665511 -0.4149617  0.85665417 -2.38058111 -0.96355898]
 [ 2.11236587  0.49579807 -0.28883991 -0.19632485 -0.92836642]
 [-11.67902098 -0.08083637 -0.56781427  2.57690596  1.8919254 ]]
```

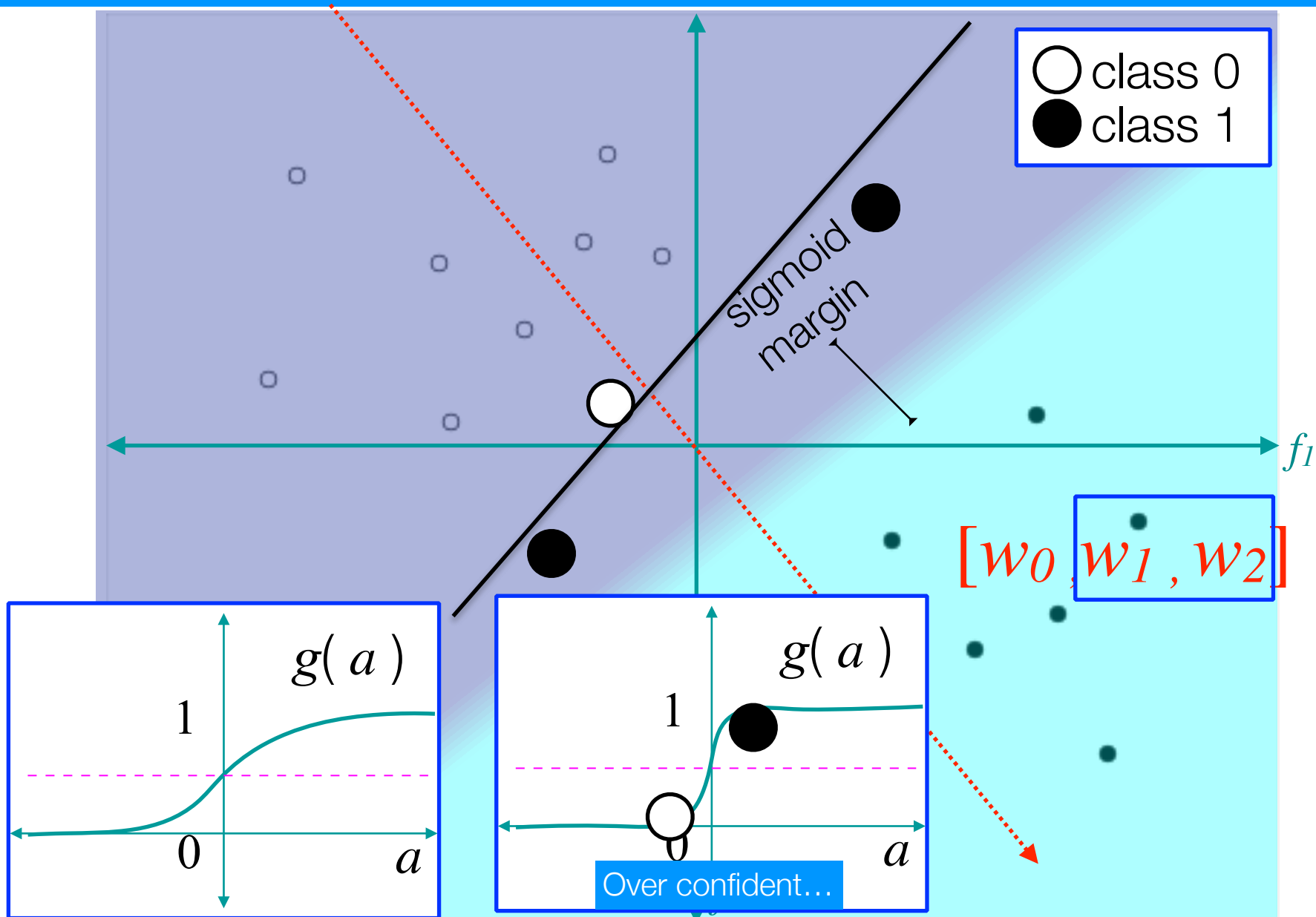
Accuracy of: 1.0

CPU times: user 17.5 ms, sys: 3.79 ms, total: 21.3 ms

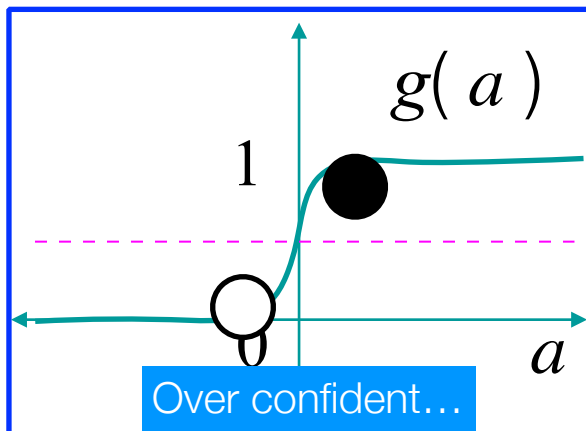
Wall time: 19.5 ms

Sklearn

Regularization, Intuition



Regularization, Mathematics



Large values of $\mathbf{w}$ allow sigmoid to be steep

$$l(\mathbf{w})_{reg} = \underbrace{l(\mathbf{w})_{prev}}_{\text{old objective}} - \underbrace{C \cdot \sum_j w_j^2}_{\text{regularization}}$$

$$\left[\left(\sum_{i=1}^M (y^{(i)} - g(\mathbf{x}^{(i)} \cdot \mathbf{w})) x_j^{(i)} \right) - \underbrace{C \cdot 2w_j}_{\text{new}} \right] \quad \mathbf{w} \leftarrow \mathbf{w} + \eta \left[\underbrace{\nabla l(\mathbf{w})_{prev}}_{\text{previous gradient}} - C \cdot 2\mathbf{w} \right]$$

```
class RegularizedBLR(VectorBLR):
    def __init__(self, C=0.0, **kwargs):
        self.C = C
        super().__init__(**kwargs) # call parent

    def _get_gradient(self, X, y):
        gradient = super()._get_gradient(X, y)
        gradient[1:] += -2 * self.C * self.w_[1:]
        return gradient
```

C is a hyperparameter controlling how much we want to optimize the previous objective function versus keep the weights small.

Beyond the Gradient

How much computation (i.e., how many multiplies) are required for calculating the gradient of:

$$\left(\frac{1}{M} \left[\sum_{i=1}^M (y^{(i)} - g(\mathbf{x}^{(i)} \cdot \mathbf{w})) \cdot \mathbf{x}^{(i)} \right] - 2C \cdot \mathbf{w} \right)$$

Where M is the number of instances and N is the number of elements in $\mathbf{w}$.

- A) $M \cdot N + 1$
- B) $2 \cdot M \cdot N + N$
- C) $2N$
- D) $2N - M$

Stochastic Gradient Descent (SGD)

- Sometimes the gradient calculation is too computational:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \left(\frac{1}{M} \left[\sum_{i=1}^M (y^{(i)} - g(\mathbf{x}^{(i)} \cdot \mathbf{w})) \mathbf{x}^{(i)} \right] - 2C \cdot \mathbf{w} \right)$$

- Approximate the gradient using a smaller number of instances (perhaps even just one instance)

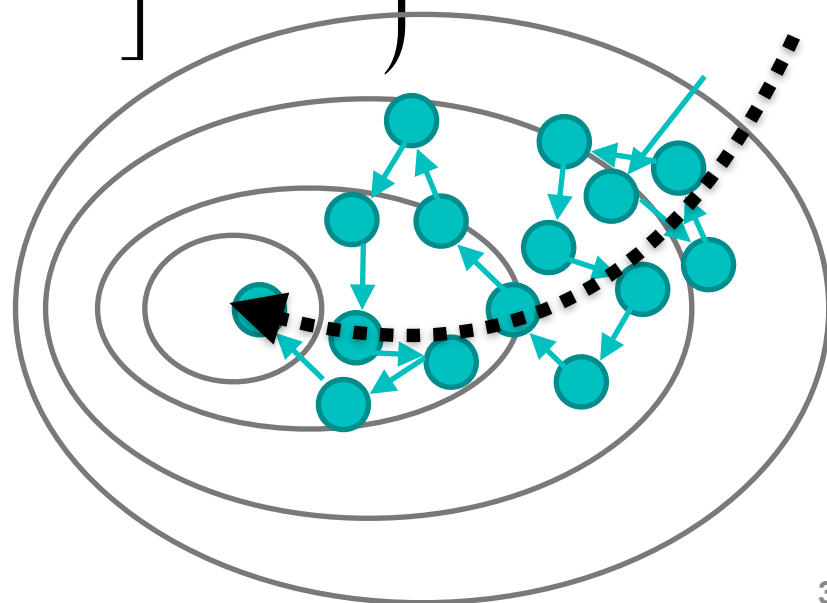
$$\mathbf{w} \leftarrow \mathbf{w} + \eta \left(\frac{1}{M_{sub}} \left[\sum_{i \in M_{sub}} (y^{(i)} - g(\mathbf{x}^{(i)} \cdot \mathbf{w})) \mathbf{x}^{(i)} \right] - 2C \cdot \mathbf{w} \right)$$

Many fewer multiplies per update

$$M_{sub} \ll M$$

$$2 \cdot M_{sub} \cdot N + N$$

But... updates can be random



Can we do better?

- When we have a function $l(w)$ where w is a scalar. The optimal value of w is given by:

$$w_{optimal} \leftarrow w_{start} - \underbrace{\left[\frac{\partial^2}{\partial w^2} l(w_{start}) \right]^{-1}}_{\text{inverse 2nd deriv}} \underbrace{\frac{\partial}{\partial w} l(w_{start})}_{\text{derivative}}$$

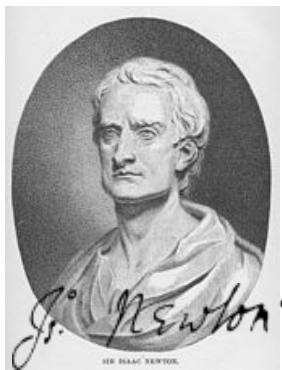
- Note that if $l(w)$ is a quadratic function, this solution converges in a single step!

When your friend thinks they found a local minimum



Newton extended this to function of multiple variables

I do not know what I may appear to the world, but to myself I seem to have been only like a boy playing on the sea-shore, and diverting myself in, now and then, finding a smoother pebble or a prettier shell than ordinary, whilst the great ocean of truth lay all undiscovered before me.



Newton's Method, the Hessian

Function of single variable: $w \leftarrow w - \underbrace{\left[\frac{\partial^2}{\partial w} l(w)\right]^{-1}}_{\text{inverse 2nd deriv}} \underbrace{\frac{\partial}{\partial w} l(w)}_{\text{derivative}}$

Function multivariate: $\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot \underbrace{\mathbf{H}(\mathbf{w})^{-1}}_{\text{inverse Hessian}} \cdot \underbrace{\nabla l(\mathbf{w})}_{\text{gradient}}$

$$\nabla^2 l(\mathbf{w}) = \mathbf{H}(\mathbf{w}) = \begin{bmatrix} \frac{\partial^2}{\partial w_1} l(\mathbf{w}), & \frac{\partial}{\partial w_1} \frac{\partial}{\partial w_2} l(\mathbf{w}), & \dots & \frac{\partial}{\partial w_1} \frac{\partial}{\partial w_{N-1}} l(\mathbf{w}) & \frac{\partial}{\partial w_1} \frac{\partial}{\partial w_N} l(\mathbf{w}) \\ \frac{\partial}{\partial w_2} \frac{\partial}{\partial w_1} l(\mathbf{w}), & \frac{\partial^2}{\partial w_2} l(\mathbf{w}), & \dots & \frac{\partial}{\partial w_2} \frac{\partial}{\partial w_{N-1}} l(\mathbf{w}) & \frac{\partial}{\partial w_2} \frac{\partial}{\partial w_N} l(\mathbf{w}) \\ & \vdots & \ddots & & \vdots \\ \frac{\partial}{\partial w_N} \frac{\partial}{\partial w_1} l(\mathbf{w}), & \frac{\partial}{\partial w_N} \frac{\partial}{\partial w_2} l(\mathbf{w}), & \dots & \frac{\partial}{\partial w_N} \frac{\partial}{\partial w_{N-1}} l(\mathbf{w}) & \frac{\partial^2}{\partial w_N} l(\mathbf{w}) \end{bmatrix}$$

$$\mathbf{H}_{j,k}(\mathbf{w}) = \frac{\partial}{\partial w_k} \left[\frac{\partial}{\partial w_j} l(\mathbf{w}) \right] \longrightarrow \frac{\partial}{\partial w_j} l(\mathbf{w}) = \sum_i (y^{(i)} - g(\mathbf{x}^{(i)} \cdot \mathbf{w})) x_j^{(i)}$$

$$\mathbf{H}_{j,k}(\mathbf{w}) = \frac{\partial}{\partial w_k} \sum_i (y^{(i)} - g(\mathbf{x}^{(i)} \cdot \mathbf{w})) x_j^{(i)}$$

$$= \sum_i \frac{\partial}{\partial w_k} y^{(i)} x_j^{(i)} - \sum_i \frac{\partial}{\partial w_k} g(\mathbf{x}^{(i)} \cdot \mathbf{w}) x_j^{(i)}$$

no dependence on w_k , zero

$$= - \sum_i x_j^{(i)} \left[\frac{\partial}{\partial w_k} g(\mathbf{x}^{(i)} \cdot \mathbf{w}) \right]$$

already know this as $g(1-g)x_k$

$$\mathbf{H}_{j,k}(\mathbf{w}) = - \sum_{i=1}^M [g(\mathbf{x}^{(i)} \cdot \mathbf{w})(1 - g(\mathbf{x}^{(i)} \cdot \mathbf{w}))] \cdot x_k^{(i)} x_j^{(i)} \quad \text{for each } j,k \text{ pair}$$

Fast Hessian Calculation

$$\mathbf{H}_{j,k}(\mathbf{w}) = - \sum_{i=1}^M [g(\mathbf{x}^{(i)} \cdot \mathbf{w})(1 - g(\mathbf{x}^{(i)} \cdot \mathbf{w}))] \cdot x_k^{(i)} x_j^{(i)} \quad \text{for each } j,k \text{ pair}$$

the sum of terms $x_k^{(i)} x_j^{(i)}$ can be calculated as follows

$$\begin{bmatrix} \sum_{i=1}^M x_1^{(i)} x_1^{(i)}, & \sum_{i=1}^M x_1^{(i)} x_2^{(i)}, & \dots & \sum_{i=1}^M x_1^{(i)} x_N^{(i)}, \\ \sum_{i=1}^M x_2^{(i)} x_1^{(i)}, & \sum_{i=1}^M x_2^{(i)} x_2^{(i)}, & \dots & \sum_{i=1}^M x_2^{(i)} x_N^{(i)}, \\ \vdots & & & \\ \sum_{i=1}^M x_N^{(i)} x_1^{(i)}, & \sum_{i=1}^M x_N^{(i)} x_2^{(i)}, & \dots & \sum_{i=1}^M x_N^{(i)} x_N^{(i)} \end{bmatrix} = \mathbf{X}^T \cdot \mathbf{X} = \mathbf{X}^T \cdot \mathbf{I} \cdot \mathbf{X}$$

$$= \begin{bmatrix} \uparrow & \uparrow & & \uparrow \\ \mathbf{x}^{(1)} & \mathbf{x}^{(2)} & \dots & \mathbf{x}^{(M)} \\ \downarrow & \downarrow & & \downarrow \end{bmatrix}_{(N \times M)} \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix}_{(M \times M)} \begin{bmatrix} \leftarrow & \mathbf{x}^{(1)} & \rightarrow \\ \leftarrow & \mathbf{x}^{(2)} & \rightarrow \\ \vdots & \vdots & \vdots \\ \leftarrow & \mathbf{x}^{(M)} & \rightarrow \end{bmatrix}_{(M \times N)}$$

we can revisit the calculation of the Hessian and use matrix operations to define the needed multiplication in an exploded view of the operations

$$\mathbf{H}[\mathbf{w}] = \begin{bmatrix} \uparrow & \uparrow & & \uparrow \\ \mathbf{x}^{(1)} & \mathbf{x}^{(2)} & \dots & \mathbf{x}^{(M)} \\ \downarrow & \downarrow & & \downarrow \end{bmatrix} \begin{bmatrix} g(\mathbf{x}^{(1)} \cdot \mathbf{w})[1 - g(\mathbf{x}^{(1)} \cdot \mathbf{w})] & \dots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \dots & g(\mathbf{x}^{(M)} \cdot \mathbf{w})[1 - g(\mathbf{x}^{(M)} \cdot \mathbf{w})] \end{bmatrix} \begin{bmatrix} \leftarrow & \mathbf{x}^{(1)} & \rightarrow \\ \leftarrow & \mathbf{x}^{(2)} & \rightarrow \\ \vdots & \vdots & \vdots \\ \leftarrow & \mathbf{x}^{(M)} & \rightarrow \end{bmatrix}$$

$$\mathbf{H}[\mathbf{w}] = \mathbf{X}^T \cdot \text{diag} [g(\mathbf{X} \cdot \mathbf{w}) \odot (1 - g(\mathbf{X} \cdot \mathbf{w}))] \cdot \mathbf{X}$$

New update direction: $\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot \underbrace{\left[\mathbf{X}^T \cdot \text{diag} [g(\mathbf{X} \cdot \mathbf{w}) \odot (1 - g(\mathbf{X} \cdot \mathbf{w}))] \cdot \mathbf{X} \right]^{-1}}_{\text{inverse Hessian}} \cdot \underbrace{\mathbf{X} \odot y_{\text{diff}}}_{\text{gradient}}$

Hessian programming

```
class HessianBinaryLogisticRegression(BLR):
    # just redo direction function
    def _get_direction(self, X, y):
        g = self.predict_proba(X, add_bias=False)
        hessian = X.T @ np.diag(g*(1-g)) @ X
        gradient = self._get_gradient(X, y)
        return pinv(hessian) @ gradient
```

Binary Logistic Regression Object with coefficients:

```
[[-0.19192085]
 [-0.00125164]
 [ 0.03170676]
 [-0.002607 ]
 [ 0.0731389 ]]
```

Hessian, Binary

Accuracy of: 0.8666666666666667

CPU times: user 1.27 ms, sys: 1.11 ms, total: 2.37 ms
Wall time: 4.44 ms

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot \underbrace{\left[\mathbf{X}^T \cdot \text{diag} [g(\mathbf{X} \cdot \mathbf{w}) \odot (1 - g(\mathbf{X} \cdot \mathbf{w}))] \cdot \mathbf{X} \right]^{-1}}_{\text{inverse Hessian}} \cdot \underbrace{\mathbf{X} \odot y_{\text{diff}}}_{\text{gradient}}$$

```
lr = MultiClassLogisticRegression(eta=50.0,
                                   iterations=20,
                                   C=0.0,
                                   solver=HessianBinaryLogisticRegression
                                   )
lr.fit(X_train, y_train)
```

```
lr_sk = LogisticRegression(solver='lbfgs',
                           C = np.inf,
                           max_iter=20) # all
# note that sklearn is optimized for using th
# ...and its faster than our implementation h
lr_sk.fit(X_train, y_train) # no need to add
```

MultiClass Logistic Regression Object with coefficients:

```
[[-5.70420524  1.68548675  1.71728225 -9.17153356 -3.17086217]
 [-0.86722628 -0.40201342 -1.0704752  3.58134621 -3.04194971]
 [-18.43258088 -0.15820534 -2.95982253  15.38047795  10.50512266]]
```

Accuracy of: 0.9666666666666667

CPU times: user 16.6 ms, sys: 4.27 ms, total: 20.9 ms

Wall time: 6.46 ms

```
[[-6.04613316  4.18486664 -9.72680649 -8.94431158]
 [ 2.95106109 -0.43710195 -3.85815241 -1.35579337]
 [ 3.09507206 -3.74776469 13.58495889 10.30010495]]
```

Accuracy of: 0.9333333333333333

CPU times: user 15.9 ms, sys: 3.01 ms, total: 18.9 ms

Wall time: 5.77 ms

Ours: Hessian, Multiclass

sklearn, Multiclass (fortran)

Logistic Regression on Titanic Data



05. Logistic Regression Dataset.ipynb

Lab 2, Town Hall



Tyler Rablin @Mr_Rablin · 2d



You're not grading assignments.

You're collecting evidence to determine student progress and pointing them towards their next steps.

Make the mental switch. It matters.

$$L_2 = C \sum_j w_j^2$$

penalty = 'l2'

$$L_1 = C \sum_j |w_j|$$

penalty = 'l1'

$$L_{1+2} = C_1 \sum_j |w_j| + C_2 \sum_j w_j^2$$

penalty = 'elasticnet'

Warning: The choice of the algorithm depends on the penalty chosen. Supported penalties by solver:

- 'lbfgs' - ['l2', None]
- 'liblinear' - ['l1', 'l2']
- 'newton-cg' - ['l2', None]
- 'newton-cholesky' - ['l2', None]
- 'sag' - ['l2', None]
- 'saga' - ['elasticnet', 'l1', 'l2', None]

Back Up Slides

Beyond the Hessian. (if time)



Solution:

Approximate the Hessian and iteratively **update** our guess, try to **eliminate** the need to find an **inverse**

Approximation of Hessian, Rank One

$$\underbrace{\mathbf{H}_{k+1}}_{\text{approx. Hessian}} \cdot \underbrace{(\mathbf{w}_{k+1} - \mathbf{w}_k)}_{\text{Change in } \mathbf{w}} = \underbrace{\nabla l(\mathbf{w}_{k+1}) - \nabla l(\mathbf{w}_k)}_{\text{Change in gradient}} \quad \textbf{Secant Property}$$

new guess guess update, low rank

$$\mathbf{H}_{k+1} = \mathbf{H}_k + \alpha_k \mathbf{u}_k \cdot \mathbf{u}_k^T$$

Each new guess $k \rightarrow k + 1$ of $\mathbf{w}_{k+1}$ allows for new Hessian Approx.

Rank One Hessian Guess

$$\underbrace{\mathbf{H}_{k+1} \mathbf{s}_k = \mathbf{v}_k}_{\text{secant property}} \rightarrow (\mathbf{H}_k + \alpha_k \mathbf{u}_k \cdot \mathbf{u}_k^T) \mathbf{s}_k = \mathbf{v}_k \quad \text{plug in and solve for } \mathbf{u}_k \text{ and } \alpha_k$$

$$\mathbf{H}_{k+1}^{-1} = \mathbf{H}_k^{-1} - \frac{\mathbf{H}_k^{-1} \mathbf{u}_k \cdot \mathbf{u}_k^T \mathbf{H}_k^{-1}}{1 + \mathbf{u}_k^T \mathbf{H}_k^{-1} \mathbf{u}_k} \quad \text{Rank One Hessian Inverse}$$

Develop Iterative Algorithm Keep Updating Quasi-Hessian

Approximation of Hessian, Rank Two

$$\underbrace{\mathbf{H}_{k+1}}_{\text{approx.}} \cdot \underbrace{(\mathbf{w}_{k+1} - \mathbf{w}_k)}_{\mathbf{s}_k} = \underbrace{\nabla l(\mathbf{w}_{k+1}) - \nabla l(\mathbf{w}_k)}_{\mathbf{v}_k}$$

**Secant
Property**

$$\mathbf{H}_{k+1} = \mathbf{H}_k + \alpha_k \mathbf{u}_k \cdot \mathbf{u}_k^T - \beta_k \mathbf{z}_k \cdot \mathbf{z}_k^T \quad \text{Rank Two Hessian}$$

$$\mathbf{H}_{k+1} \mathbf{s}_k = \mathbf{v}_k \quad \rightarrow \quad (\mathbf{H}_k + \alpha_k \mathbf{u}_k \cdot \mathbf{u}_k^T - \beta_k \mathbf{z}_k \cdot \mathbf{z}_k^T) \mathbf{s}_k = \mathbf{v}_k$$

$$\mathbf{u}_k = \mathbf{v}_k \quad \mathbf{z}_k = \mathbf{H}_k \mathbf{s}_k \quad \alpha_k = \frac{1}{\mathbf{u}_k^T \mathbf{s}_k} \quad \beta_k = \frac{1}{\mathbf{z}_k^T \mathbf{s}_k} \quad \text{solve for } \mathbf{u}_k, \mathbf{z}_k, \alpha_k, \beta_k$$

$$\mathbf{H}_{k+1}^{-1} = \mathbf{H}_k^{-1} + \frac{(\mathbf{s}_k^T \mathbf{v}_k + \mathbf{H}_k^{-1})(\mathbf{s}_k \mathbf{s}_k^T)}{(\mathbf{s}_k^T \mathbf{v}_k)^2} - \frac{\mathbf{H}_k^{-1} \mathbf{v}_k \mathbf{s}_k^T + \mathbf{s}_k \mathbf{v}_k^T \mathbf{H}_k^{-1}}{\mathbf{s}_k^T \mathbf{v}_k} \quad \text{Rank Two Inverse}$$

Develop Iterative Algorithm Keep Updating Quasi-Hessian

David F. Shanno

- Started out as Engineer in Gulf Oil, but wasn't super motivated by profits ...
 - leaves for academic research
- 1967: Got into a new field, "math programming"
- Went to a lecture and thought he could improve on algorithm (conditioning on approximation)
- Wrote with Broyden, Fletcher, Goldfarb and they all thought what they were doing was different
- Turns out it was same, release a joint paper (before meeting yet), Shared the credit
- Left Chicago because good friend was murdered, wanted a safer place for kids
- Offered Presidency of Toronto University (\$\$\$\$)
- Turned it down to continue math research



1938 - 2019

